Cryptography

Symmetric Encryption • File Encryption • Cryptographic Hashing

Student
Karim Hamd

Supervised by
Eng. Saher Waleed

Date
October 2026

Online AES Symmetric Encryption & Decryption

Overview

In this task, an online tool was used to demonstrate symmetric encryption using the Advanced Encryption Standard (AES) algorithm.

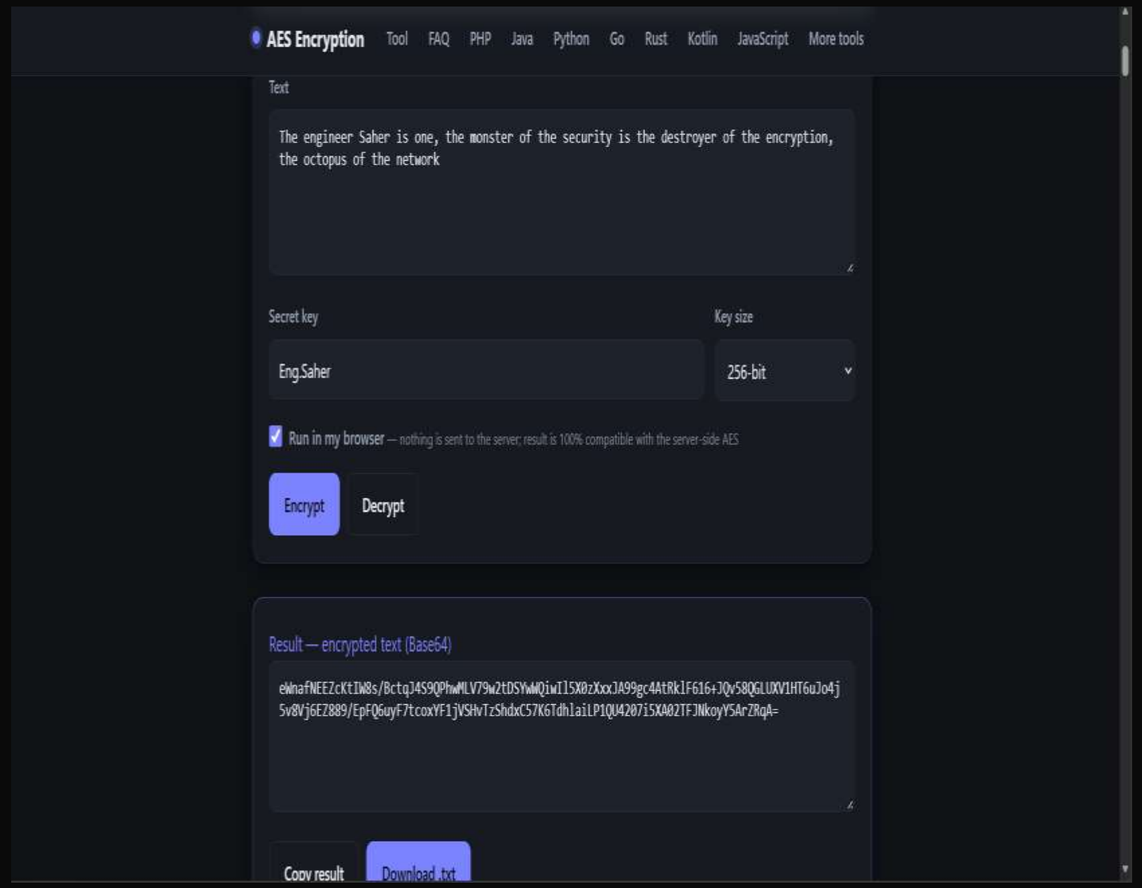
Concept — Symmetric Encryption

Symmetric encryption uses a single pre-shared secret key to both encrypt the plaintext and decrypt the ciphertext. Without the correct secret key, the encrypted message cannot be reverted back to its original plain form.

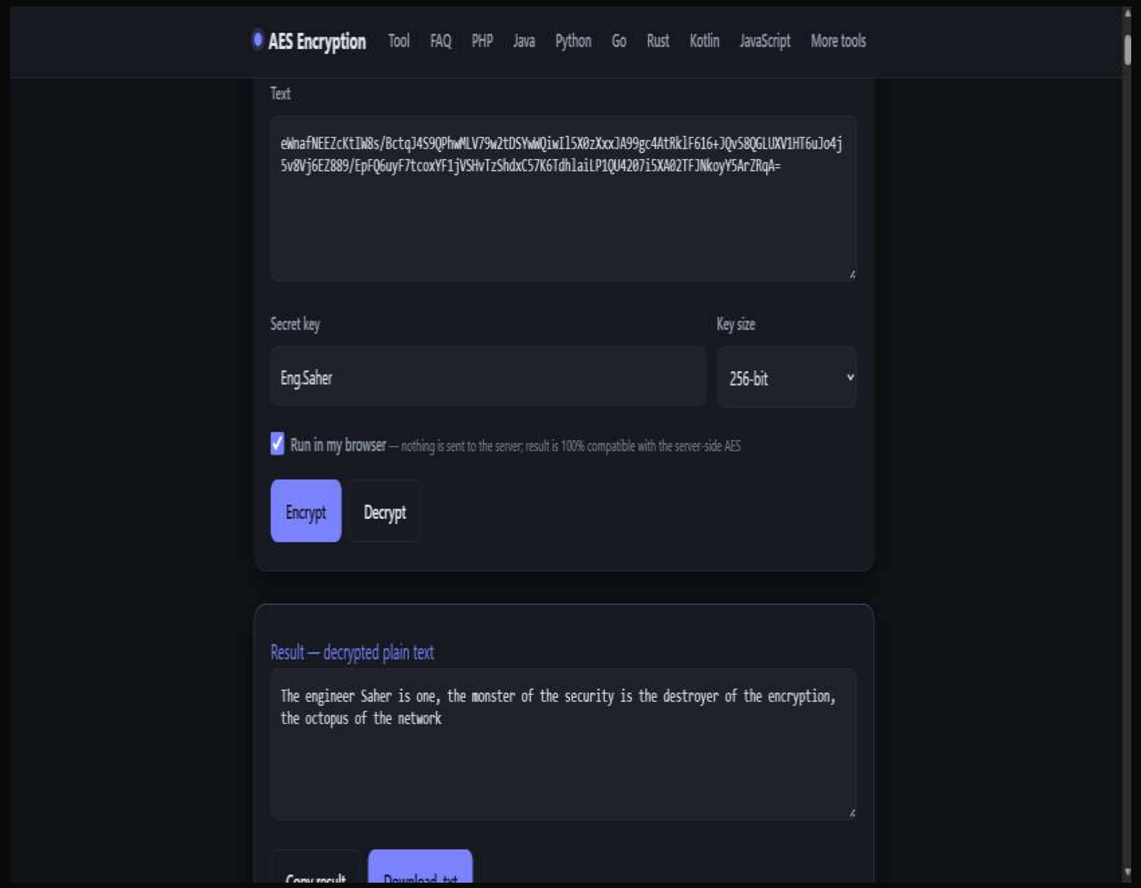
In the demonstration, the plaintext “The engineer Saher is one, the monster of the security is the destroyer of the encryption, the octopus of the network” was encrypted with the secret key “Eng.Saher” using AES-256, producing a Base64 ciphertext. The same key successfully restored the original message.

Demonstration — Encrypt & Decrypt

Encryption Result (AES-256, Key: Eng.Saher)



Decryption Result — Original Plaintext Restored



File Encryption and Decryption using OpenSSL

Overview

A text file named “SafeSurf_file.txt” containing multiple lines of text was created on Kali Linux. The file was then encrypted and decrypted using the OpenSSL CLI tool with AES-256-CBC.

Commands Executed

1. File Creation

```
nano ~/SafeSurf_file.txt
```

2. File Encryption (AES-256-CBC)

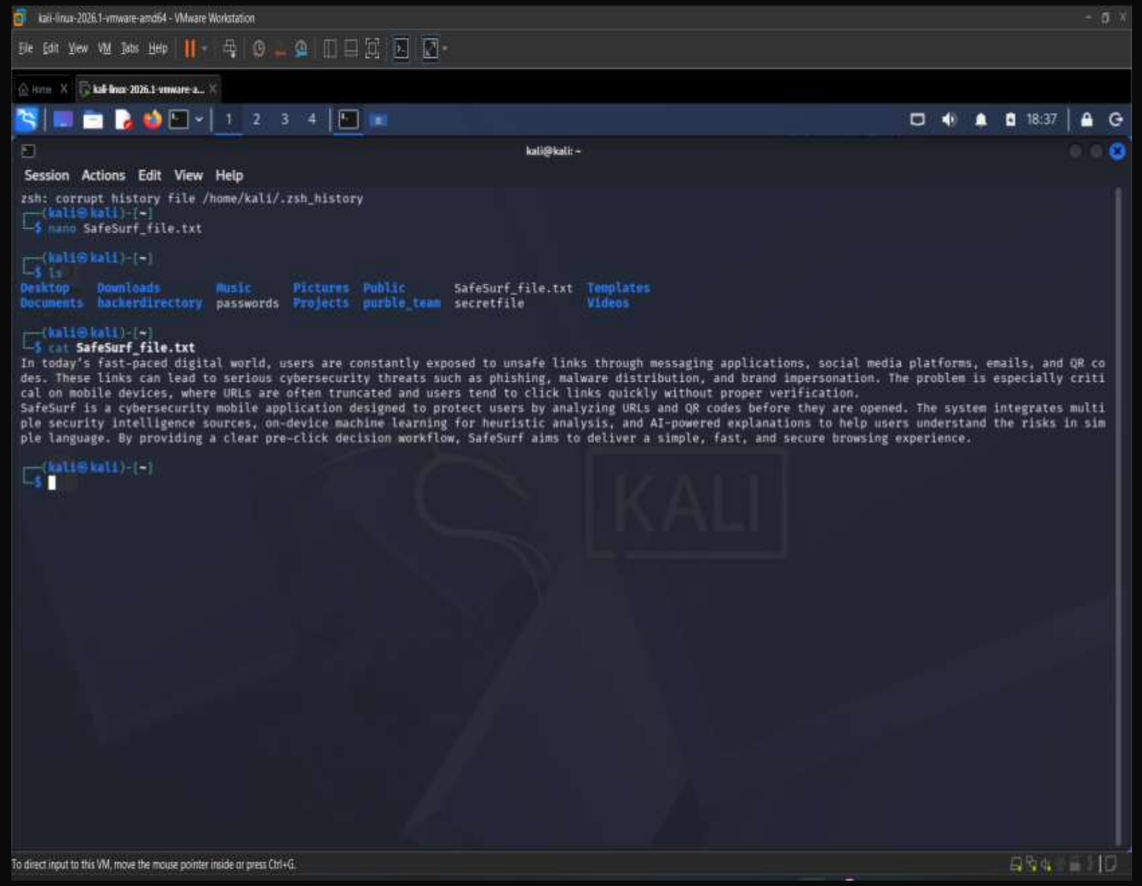
```
openssl enc -aes-256-cbc -salt -in ~/SafeSurf_file.txt -out ~/SafeSurf_file.enc
```

3. File Decryption

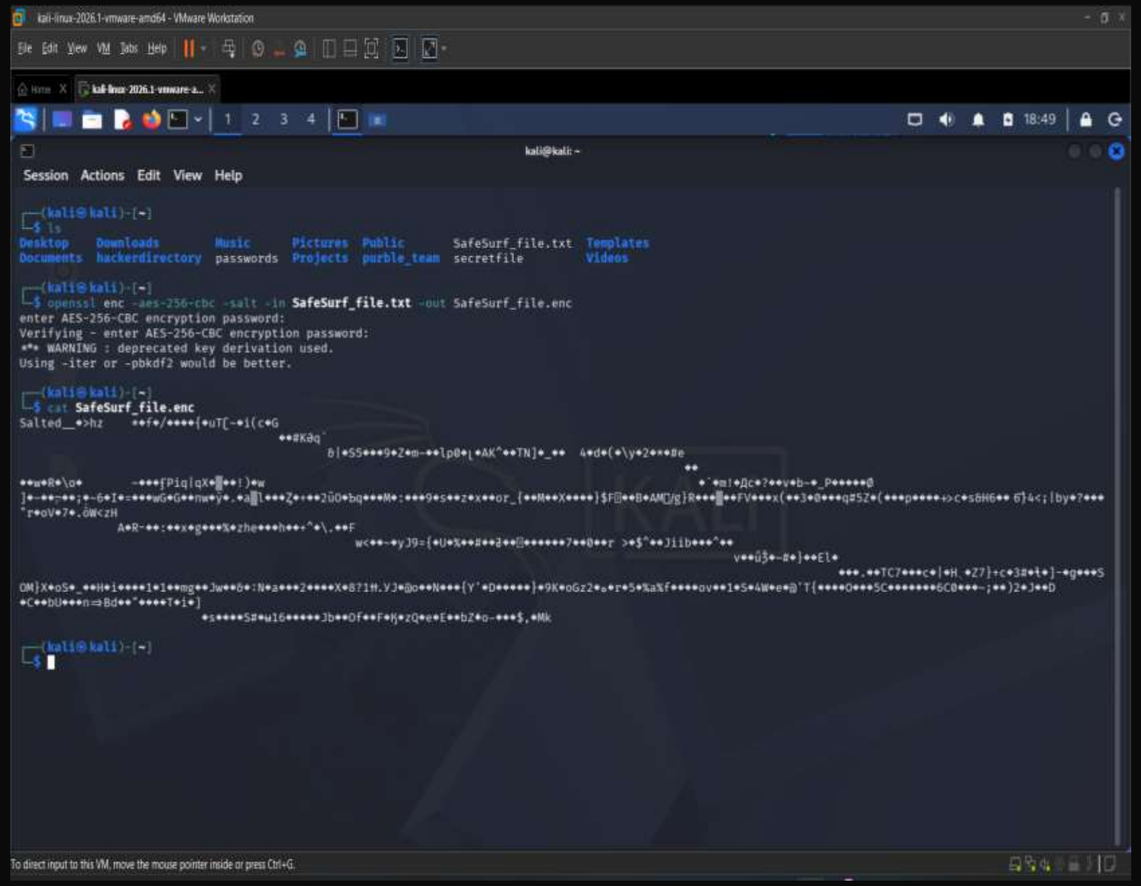
```
openssl enc -d -aes-256-cbc -in ~/SafeSurf_file.enc -out ~/decrypted_file.txt
```

File Content & Encryption

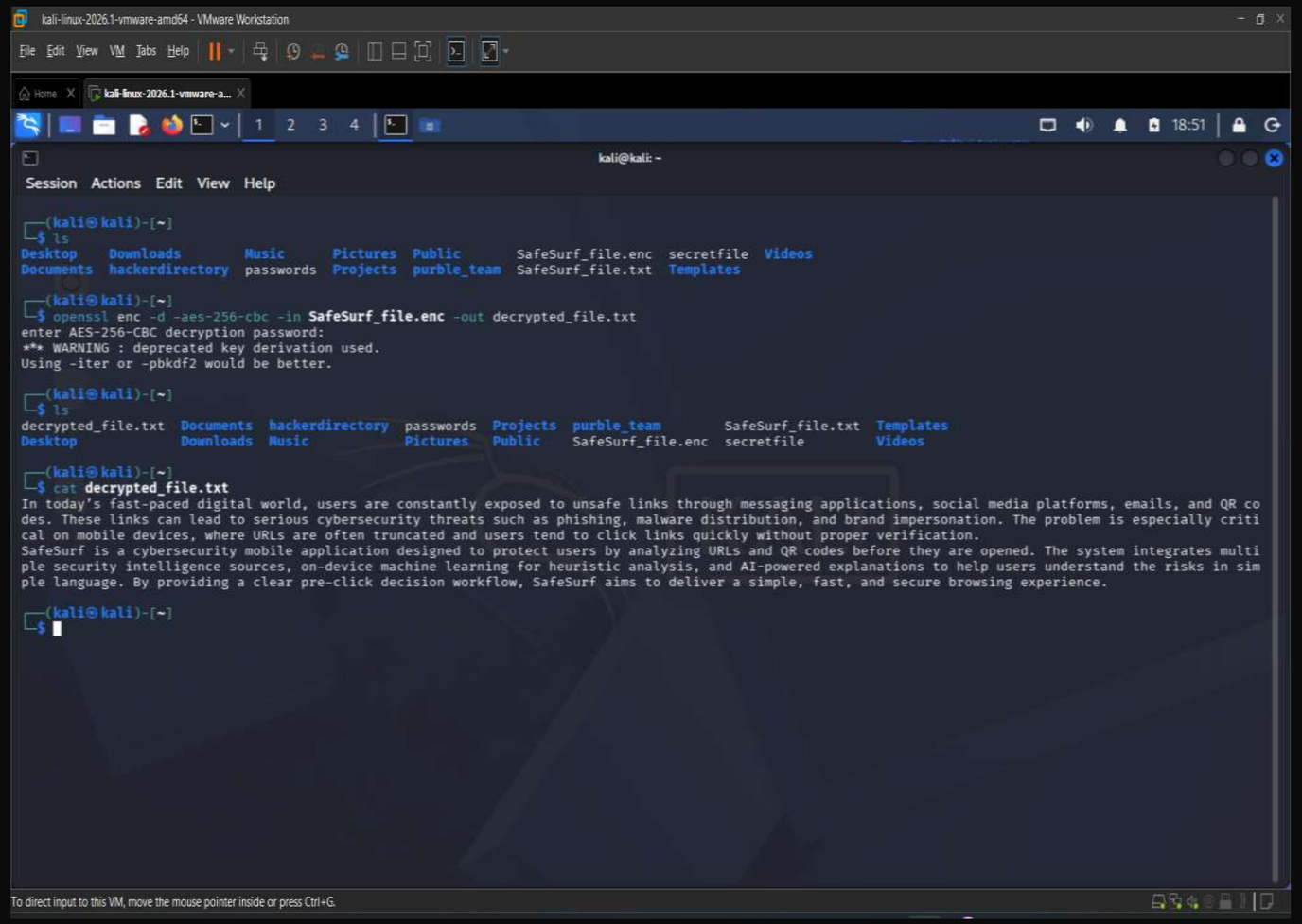
SafeSurf_file.txt (Original Content)



OpenSSL Encryption (AES-256-CBC)



Decryption Result & Concept



Concept

OpenSSL uses the AES-256-CBC algorithm along with a user-defined passphrase to transform the contents of “SafeSurf_file.txt” into a secure binary ciphertext file (“SafeSurf_file.enc”).

Decryption reverses this process using the same passphrase to ensure confidentiality. The original multi-line text was fully restored, confirming the integrity of the encryption workflow.

Generating File Hash using OpenSSL (SHA-256)

Overview

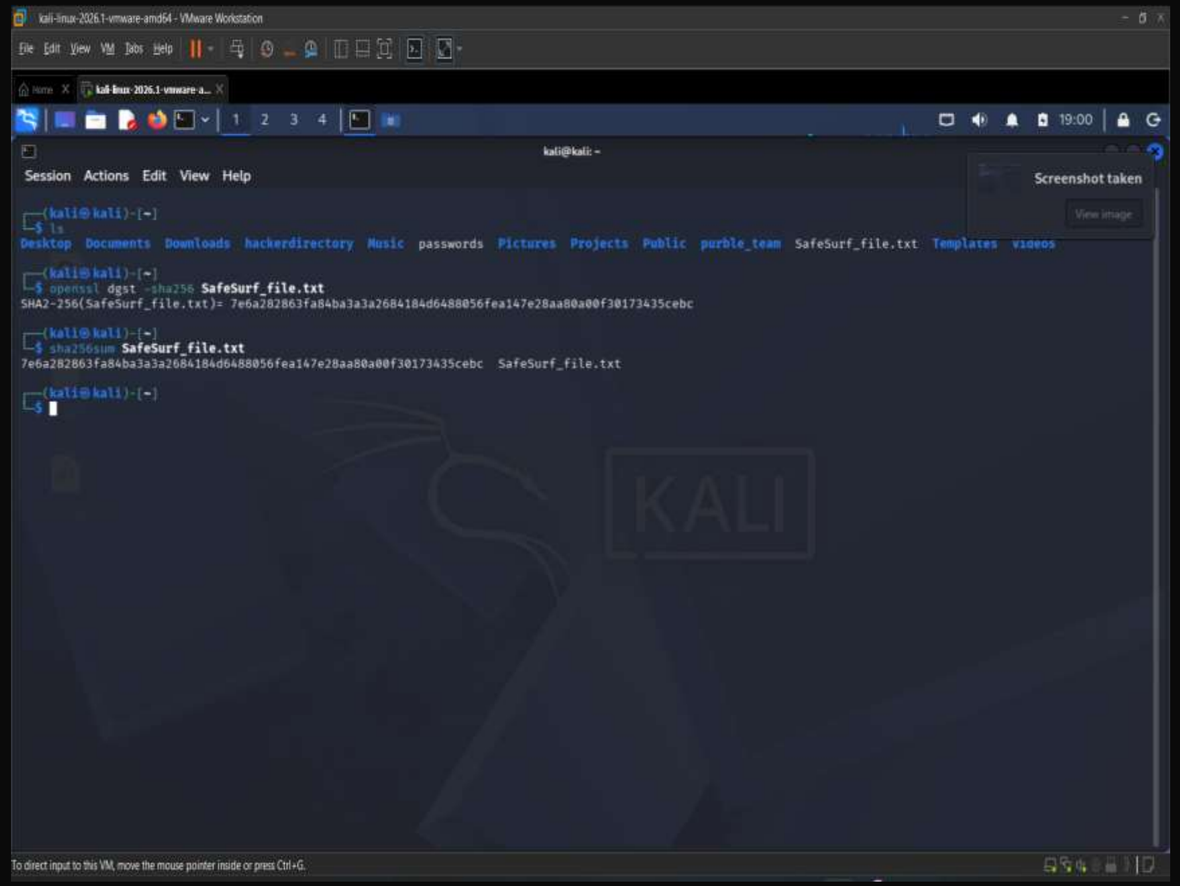
A cryptographic hash of the multi-line file “SafeSurf_file.txt” was generated using the SHA-256 hashing algorithm via OpenSSL.

Command

```
openssl dgst -sha256 ~/SafeSurf_file.txt
```

Concept & Explanation

A cryptographic hash is a one-way mathematical function that generates a fixed-length unique fingerprint (digest) of the file. Hashing is non-reversible (unlike encryption). It is used to verify Data Integrity: if even a single character inside the file is modified, the resulting SHA-256 hash output will change completely.



Tool Description: OpenSSL

What is OpenSSL?

OpenSSL is a robust, commercial-grade, and full-featured toolkit for the Transport Layer Security (TLS) and Secure Sockets Layer (SSL) protocols. It is widely used in Linux distributions (such as Kali Linux) for various cryptographic tasks, including file encryption, decryption, generation of digital signatures, and cryptographic hashing.

Role in this Activity

01

Symmetric File Encryption & Decryption

Utilizing the AES-256-CBC cryptographic cipher combined with a secure passphrase to safeguard file confidentiality.

02

Cryptographic Hashing

Utilizing the SHA-256 digest function to compute a unique fingerprint of the file, ensuring data integrity.